

Next Three Steps

Step	Bug State	Responsible	Action
1	Reopened	QA Engineer	Confirm the issue still exists and reopen it with retest evidence.
2	Assigned	QA Lead / Developer	Assign the defect to the appropriate developer.
3	In Progress	Developer	Investigate the root cause, correct the code, and prepare it for retest.

Flow: Fixed → Reopened → Assigned → In Progress → Fixed → Retest

Question 2 — Severity and Priority

Defect (a): Place Order button becomes unresponsive after coupon application and blocks transactions.

Severity: S1 — Critical

Priority: P1 — High

Justification: It blocks a core business function and prevents customers from completing orders.

Defect (b): Company logo is misaligned by 4 pixels on checkout.

Severity: S4 — Minor

Priority: P3 — Low

Justification: It is a visual issue and does not prevent checkout.

High Severity but Low Priority: Severity describes impact on the system, while priority describes how urgently the team needs to fix it. A serious problem can have lower priority when it affects a rarely used feature or a feature temporarily deprioritized by the business.

Question 3 — Selenium IDE Automation

Criterion 1 — Repetitive Testing: The regression suite runs twice every sprint, so automation reduces repetitive manual execution.

Criterion 2 — Time Saving: Manual regression takes about 4 hours per cycle; automation can reduce repeated browser actions.

Criterion 3 — Frequent Updates: The search/filter feature changes every sprint, making repeatable automated regression useful.

Limitations: (1) Selenium IDE is less suitable for highly complex automation logic. (2) Tests require maintenance when locators, DOM structure, or workflows change.

Question 4 — Locator Strategy

Use the stable `id=btn-confirm-order` locator because the ID remains stable across releases, while the XPath depends on the DOM structure.

```
//div[@class='checkout-wrapper']/button[3]
```

A **fragile locator** depends on details likely to change, such as DOM hierarchy, element position, dynamic classes, or indexes. A fragile locator can make a regression test fail even when the application works, creating false failures and maintenance effort.

Question 5 — JOIN Validation

```
SELECT
u.name AS customer_name,
r.name AS restaurant_name,
o.amount AS order_amount
FROM orders o
INNER JOIN users u
ON o.user_id = u.user_id
INNER JOIN restaurants r
ON o.restaurant_id = r.restaurant_id;
```

A LEFT JOIN can be more appropriate because it keeps all orders even when a matching user or restaurant record is missing. This can reveal an order with a NULL customer or restaurant name, indicating a possible data-integrity problem.

```
SELECT
o.order_id,
u.name AS customer_name,
r.name AS restaurant_name,
o.amount
FROM orders o
LEFT JOIN users u ON o.user_id = u.user_id
LEFT JOIN restaurants r ON o.restaurant_id = r.restaurant_id;
```

Question 6 — ACID Property

The violated property is **Atomicity**. The wallet debit and order creation should behave as one complete transaction. If the wallet is debited but the order is not created, only part of the transaction completed.

```
SELECT user_id, amount, transaction_type, transaction_status
FROM wallet_transactions
WHERE user_id = 101
AND transaction_type = 'DEBIT'
ORDER BY transaction_id DESC;
```

```
SELECT order_id, user_id, amount, status
FROM orders
WHERE user_id = 101
ORDER BY order_id DESC;
```

ACID compliance is important because it helps prevent financial discrepancies, incorrect application state, and customer-impacting transaction failures.

SECTION B — PRACTICAL TASKS

TASK 1 — DEFECT REPORT

Defect ID: BUG-001

Title: Checkout button becomes unresponsive when cart contains more than 10 items.

Module: Checkout / Cart

Severity: S1 — Critical

Priority: P1 — High

Severity justification: The defect prevents users from proceeding to checkout when the cart contains more than 10 items.

Priority justification: It affects a core purchase flow and requires immediate investigation.

Preconditions: User is logged in; application is available; at least 11 valid menu items are available.

Steps to Reproduce:

1. Open the food delivery application.
2. Log in with valid credentials.
3. Search for a restaurant.
4. Open the restaurant menu.
5. Add more than 10 items to the cart.
6. Open the cart.
7. Click Checkout.

Expected Result: The checkout page should open successfully.

Actual Result: The Checkout button becomes unresponsive and the user cannot proceed.

Environment: Google Chrome, Windows 11, Food Delivery Web App, example build v1.0.0.

Bug Life Cycle: New.

Retest Test Case — TC-RET-001

1. Open the application.
2. Log in with valid credentials.
3. Add more than 10 items to the cart.
4. Open the cart and verify all items.
5. Click Checkout.

Expected: Checkout opens and displays selected items, total amount, and checkout options.

TASK 2 — SELENIUM IDE TEST SUITE

Suite Name: FoodDelivery_Regression_Suite

Execution Order: TC-SEL-001, then TC-SEL-002

Test Case 1 — Valid Login and Add to Cart

Step	Command	Target	Value
1	open	/	
2	type	id=email	testuser@example.com
3	type	id=password	Test@123
4	click	id=login-button	
5	waitForElementVisible	id=search-box	10000
6	type	id=search-box	Pizza Palace
7	click	id=search-button	

Step	Command	Target	Value
8	waitForElementVisible	css=.restaurant-result	10000
9	click	css=.restaurant-result	
10	waitForElementVisible	id=menu-item-1	10000
11	click	id=menu-item-1	
12	click	id=add-to-cart	
13	verifyElementPresent	id=cart	

XPath example if needed: //button[contains(@class,'add-to-cart')]

Test Case 2 — Invalid Login

Step	Command	Target	Value
1	open	/	
2	type	id=email	invalid@example.com
3	type	id=password	Wrong@123
4	click	id=login-button	
5	waitForElementVisible	id=login-error	10000
6	verifyText	id=login-error	Invalid email or password

TASK 3 — SQL BACKEND VALIDATION

3.1 Verify Newly Placed Order

```
SELECT order_id, user_id, total_amount, status
FROM orders
WHERE user_id = 101
AND total_amount = 499.00
AND status = 'placed';
```

3.2 INNER JOIN — Orders, Users and Restaurants

```
SELECT o.order_id, u.user_id, u.name AS customer_name,
r.restaurant_id, r.name AS restaurant_name,
o.amount, o.status
FROM orders o
INNER JOIN users u ON o.user_id = u.user_id
INNER JOIN restaurants r ON o.restaurant_id = r.restaurant_id;
```

3.3 Insert Test User

```
INSERT INTO users (user_id, name, email, city)
VALUES (1001, 'Test User', 'testuser1001@example.com', 'Ahmedabad');

SELECT user_id, name, email, city
FROM users
WHERE email = 'testuser1001@example.com';
```

3.4 Identify Invalid Order Amounts

```
SELECT order_id, user_id, total_amount, status
FROM orders
WHERE total_amount IS NULL
OR total_amount = 0;
```

TASK 4 — END-TO-END QA WORKFLOW

Selenium sequence: open → login → search restaurant → waitForElementVisible → select restaurant → add item → apply coupon → place order → assertText order confirmation.

```
SELECT order_id, user_id, coupon_code,  
discount_applied, final_amount  
FROM orders  
WHERE user_id = 101  
AND coupon_code = 'SAVE50';
```

```
SELECT o.order_id, o.user_id, o.final_amount,  
p.payment_id, p.payment_status  
FROM orders o  
LEFT JOIN payments p ON o.order_id = p.order_id  
WHERE o.user_id = 101  
AND p.payment_id IS NULL;
```

Example defect — BUG-002: Order is created successfully but the corresponding payment record is not created.

Severity: S1 — Critical.

Priority: P1 — High.

State: New.

SECTION C — MINI CAPSTONE PROJECT

Food Delivery App QA Suite

Scope: User registration/login, restaurant search, menu selection, cart, checkout, coupon application, order placement, payment, order history, and backend validation.

Objectives: Verify login, search, cart, checkout, coupon calculation, order placement, payment record creation, and database consistency.

Tool	Purpose
JIRA	Defect management
Selenium IDE	UI automation
MySQL	Backend/database validation

Entry Criteria: Application build available; test environment accessible; database accessible; test accounts/data available; major environment blockers resolved.

Exit Criteria: Planned tests executed; critical defects resolved or accepted; regression completed; SQL validation completed; payment/order consistency verified; results documented.

2. DEFECT LOG

ID	Defect	Severity	Priority	State
BUG-001	Checkout button unresponsive with >10 items	S1	P1	New
BUG-002	Order created without payment record	S1	P1	New
BUG-003	Restaurant name missing from order history	S2	P2	New

BUG-003 — Restaurant name is missing from order history for a valid completed order.

Steps: Login → place valid order → complete order → open Order History → check restaurant information.

Expected: Correct restaurant name displayed.

Actual: Restaurant name missing.

Severity: S2 — Major. **Priority:** P2 — Medium. **Environment:** Chrome, Windows 11, example v1.0.0. **State:** New.

3. SELENIUM IDE TEST PLAN

TC ID	Test Case	Locator Strategy	Expected Result
TC-001	Valid Login	ID / Name	User successfully logs in
TC-002	Restaurant Search	ID / CSS	Matching restaurant results displayed
TC-003	Add Item to Cart	ID / CSS	Selected item appears in cart
TC-004	Apply Coupon	ID / Name	Correct discount applied
TC-005	Confirm Order	ID + assertText	Order confirmation displayed

4. SQL VALIDATION QUERIES

```
-- Query 1: New user registration
SELECT user_id, name, email
FROM users
WHERE email = 'testuser1001@example.com';
```

```
-- Query 2: Order placement verification
SELECT u.user_id, u.name AS customer_name,
o.order_id, o.total_amount, o.status
FROM users u
INNER JOIN orders o ON u.user_id = o.user_id
WHERE u.user_id = 101
AND o.status = 'placed';

-- Query 3: Coupon discount validation
SELECT order_id, coupon_code, discount_applied, final_amount
FROM orders
WHERE user_id = 101
AND coupon_code = 'SAVE50';

-- Query 4: Payment confirmation
SELECT o.order_id, o.final_amount,
p.payment_id, p.amount, p.payment_status
FROM orders o
INNER JOIN payments p ON o.order_id = p.order_id
WHERE o.user_id = 101;

-- Query 5: Order history
SELECT o.order_id, u.name AS customer_name,
r.name AS restaurant_name, o.amount, o.status
FROM orders o
INNER JOIN users u ON o.user_id = u.user_id
INNER JOIN restaurants r ON o.restaurant_id = r.restaurant_id
WHERE o.user_id = 101
ORDER BY o.order_id DESC;
```

5. REQUIREMENT TRACEABILITY MATRIX — RTM

Req. ID	Functional Requirement	Selenium Test	Defect ID
FR-001	User can log in with valid credentials	TC-001	—
FR-002	User can search for a restaurant	TC-002	BUG-003
FR-003	User can add an item to cart	TC-003	BUG-001
FR-004	User can apply a coupon	TC-004	—
FR-005	User receives order confirmation	TC-005	BUG-002

SECTION D — AI-AUGMENTED LEARNING

STEP 1 — BUILD WITH AI

AI Prompt

Create a MySQL database named food_delivery with three tables: users, orders, and payments.

The tables must include appropriate PRIMARY KEY, FOREIGN KEY, NOT NULL, and UNIQUE constraints.

Insert at least five test records into each table.

Include at least one deliberate edge case, such as an order without a matching payment record.

Also provide three SQL validation queries using JOIN and WHERE conditions to verify that completed orders have corresponding payment records with matching amounts.

Use MySQL syntax and clearly label each section.

Original AI SQL Script

```

CREATE DATABASE food_delivery;
USE food_delivery;

CREATE TABLE users (
  user_id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE,
  phone VARCHAR(20)
);

CREATE TABLE orders (
  order_id INT PRIMARY KEY,
  user_id INT NOT NULL,
  total_amount DECIMAL(10,2) NOT NULL,
  status VARCHAR(30) NOT NULL,
  FOREIGN KEY (user_id) REFERENCES users(user_id)
);

CREATE TABLE payments (
  payment_id INT PRIMARY KEY,
  order_id INT NOT NULL,
  amount DECIMAL(10,2) NOT NULL,
  payment_status VARCHAR(30) NOT NULL,
  FOREIGN KEY (order_id) REFERENCES orders(order_id)
);

INSERT INTO users VALUES
(1, 'Amit', 'amit@example.com', '9999999991'),
(2, 'Neha', 'neha@example.com', '9999999992'),
(3, 'Rahul', 'rahul@example.com', '9999999993'),
(4, 'Priya', 'priya@example.com', '9999999994'),
(5, 'Jinal', 'jinal@example.com', NULL);

INSERT INTO orders VALUES
(101, 1, 450.00, 'completed'),
(102, 2, 500.00, 'completed'),
(103, 3, 300.00, 'placed'),
(104, 4, 750.00, 'completed'),
(105, 5, 250.00, 'completed');

INSERT INTO payments VALUES
(1001, 101, 450.00, 'success'),
(1002, 102, 500.00, 'success'),
(1003, 103, 300.00, 'success'),
(1004, 104, 750.00, 'success');

SELECT o.order_id, o.total_amount, p.amount, p.payment_status
FROM orders o INNER JOIN payments p ON o.order_id = p.order_id
WHERE o.status = 'completed';

SELECT o.order_id, o.total_amount, p.amount
FROM orders o INNER JOIN payments p ON o.order_id = p.order_id
WHERE o.status = 'completed' AND o.total_amount <> p.amount;

SELECT o.order_id, o.total_amount
FROM orders o LEFT JOIN payments p ON o.order_id = p.order_id
WHERE o.status = 'completed' AND p.payment_id IS NULL;

```

STEP 2 — TEST & DEBUG WITHOUT AI

Issue Found: The original script defined email as UNIQUE but did not explicitly define it as NOT NULL. For stronger test-data validation, email should be explicitly NOT NULL.

Corrected SQL Script

```

CREATE DATABASE IF NOT EXISTS food_delivery;
USE food_delivery;

CREATE TABLE users (
  user_id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  phone VARCHAR(20)
);

CREATE TABLE orders (
  order_id INT PRIMARY KEY,
  user_id INT NOT NULL,
  total_amount DECIMAL(10,2) NOT NULL,
  status VARCHAR(30) NOT NULL,
  FOREIGN KEY (user_id) REFERENCES users(user_id)
);

CREATE TABLE payments (
  payment_id INT PRIMARY KEY,
  order_id INT NOT NULL,
  amount DECIMAL(10,2) NOT NULL,
  payment_status VARCHAR(30) NOT NULL,
  FOREIGN KEY (order_id) REFERENCES orders(order_id)
);

INSERT INTO users (user_id,name,email,phone) VALUES
(1, 'Amit', 'amit@example.com', '9999999991'),
(2, 'Neha', 'neha@example.com', '9999999992'),
(3, 'Rahul', 'rahul@example.com', '9999999993'),
(4, 'Priya', 'priya@example.com', '9999999994'),
(5, 'Jinal', 'jinal@example.com', NULL);

INSERT INTO orders (order_id,user_id,total_amount,status) VALUES
(101,1,450.00,'completed'),
(102,2,500.00,'completed'),
(103,3,300.00,'placed'),
(104,4,750.00,'completed'),
(105,5,250.00,'completed');

INSERT INTO payments (payment_id,order_id,amount,payment_status) VALUES
(1001,101,450.00,'success'),
(1002,102,500.00,'success'),
(1003,103,300.00,'success'),
(1004,104,750.00,'success');

SELECT o.order_id,o.total_amount,p.amount,p.payment_status
FROM orders o INNER JOIN payments p ON o.order_id=p.order_id
WHERE o.status='completed';

SELECT o.order_id,o.total_amount,p.amount
FROM orders o INNER JOIN payments p ON o.order_id=p.order_id
WHERE o.status='completed' AND o.total_amount<>p.amount;

SELECT o.order_id,o.total_amount
FROM orders o LEFT JOIN payments p ON o.order_id=p.order_id
WHERE o.status='completed' AND p.payment_id IS NULL;

```

3–4 Line Explanation: The original AI-generated script did not explicitly make the email column NOT NULL. I corrected this by adding NOT NULL along with UNIQUE. This ensures every test user has an email value and prevents incomplete user records. I also used explicit column lists in INSERT statements to make the test data safer and clearer.

FINAL SUBMISSION CHECKLIST

Section	Required Item	Status
A	Concept Application Q1–Q6	Completed
B	Task 1 — Defect Report + Retest	Completed
B	Task 2 — Selenium IDE Suite	Completed
B	Task 3 — SQL Backend Validation	Completed
B	Task 4 — End-to-End QA Workflow	Completed
C	Mini Test Plan	Completed
C	3 Defect Reports / Log	Completed
C	5 Selenium Test Cases	Completed
C	5 SQL Validation Queries	Completed
C	RTM	Completed
D	AI Prompt	Completed
D	Original + Corrected SQL	Completed
D	3–4 Line Improvement Note	Completed

Submission Note: Run the SQL queries in your own MySQL environment and capture the actual query-output screenshots required by the assessment. Replace example credentials, IDs, build versions, and UI text with the actual values from your application before submission.